



Lab Task(s) Report – Lab 10

Name:

Muhammad Abu Bakar

NU ID:

24K-0826

May 10th, 2026

—

Syllabus:

Database Systems Lab

Syllabus Code: CL-2005

—

Instructor:

Sir Osama Tahir

Q1)

SQL Query Text:

```
SET SERVEROUTPUT ON;
```

DECLARE

```
v_base_salary HR.Employees.Salary%type := 250;  
v_sp_allowance_amt HR.Employees.Salary%type;  
v_f_tot_sal HR.Employees.Salary%type;
```

BEGIN

```
v_sp_allowance_amt := v_base_salary * 0.15;  
v_f_tot_sal := v_base_salary + v_sp_allowance_amt;
```

```
DBMS_OUTPUT.PUT_LINE('Base Sal: $' || v_base_salary);
```

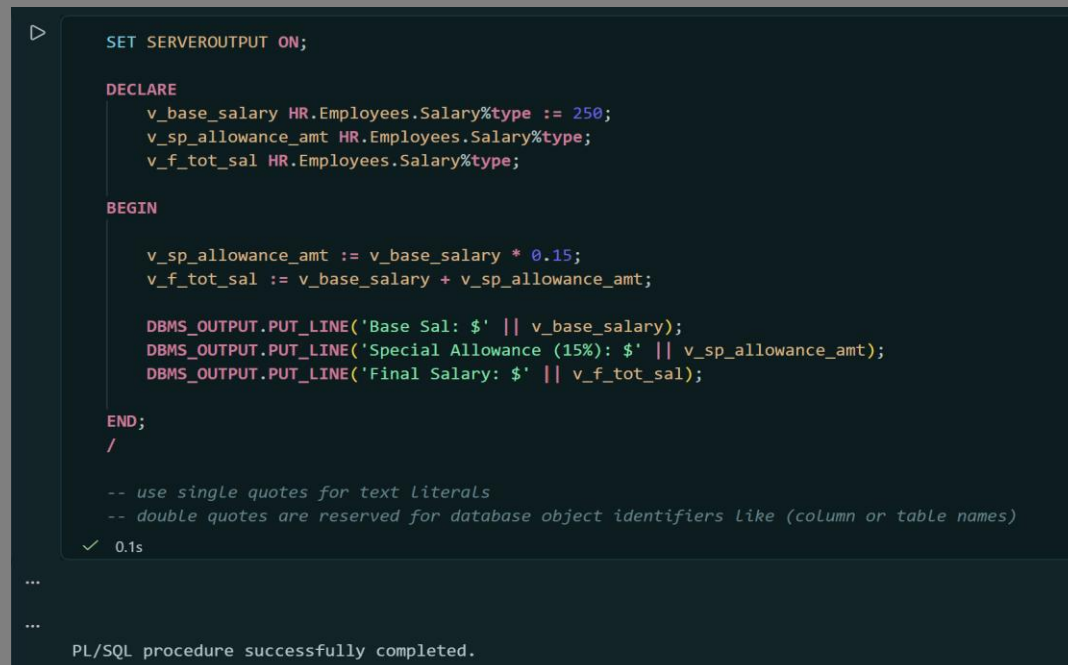
```
DBMS_OUTPUT.PUT_LINE('Special Allowance (15%): $' || v_sp_allowance_amt);
```

```
DBMS_OUTPUT.PUT_LINE('Final Salary: $' || v_f_tot_sal);
```

```
END;
```

```
/
```

Screenshot:



```
SET SERVEROUTPUT ON;  
  
DECLARE  
    v_base_salary HR.Employees.Salary%type := 250;  
    v_sp_allowance_amt HR.Employees.Salary%type;  
    v_f_tot_sal HR.Employees.Salary%type;  
  
BEGIN  
  
    v_sp_allowance_amt := v_base_salary * 0.15;  
    v_f_tot_sal := v_base_salary + v_sp_allowance_amt;  
  
    DBMS_OUTPUT.PUT_LINE('Base Sal: $' || v_base_salary);  
    DBMS_OUTPUT.PUT_LINE('Special Allowance (15%): $' || v_sp_allowance_amt);  
    DBMS_OUTPUT.PUT_LINE('Final Salary: $' || v_f_tot_sal);  
  
END;  
/  
  
-- use single quotes for text literals  
-- double quotes are reserved for database object identifiers like (column or table names)  
✓ 0.1s  
...  
...  
PL/SQL procedure successfully completed.
```

Q2)

SQL Query Text:

```
SET SERVEROUTPUT ON;
```

DECLARE

```
v_fn HR.employees.first_name%type;
```

```
v_sal HR.employees.salary%type;
```

BEGIN

```
SELECT first_name, salary
```

```
into v_fn, v_sal
```

```
from HR.employees
```

```
where employee_id = 105;
```

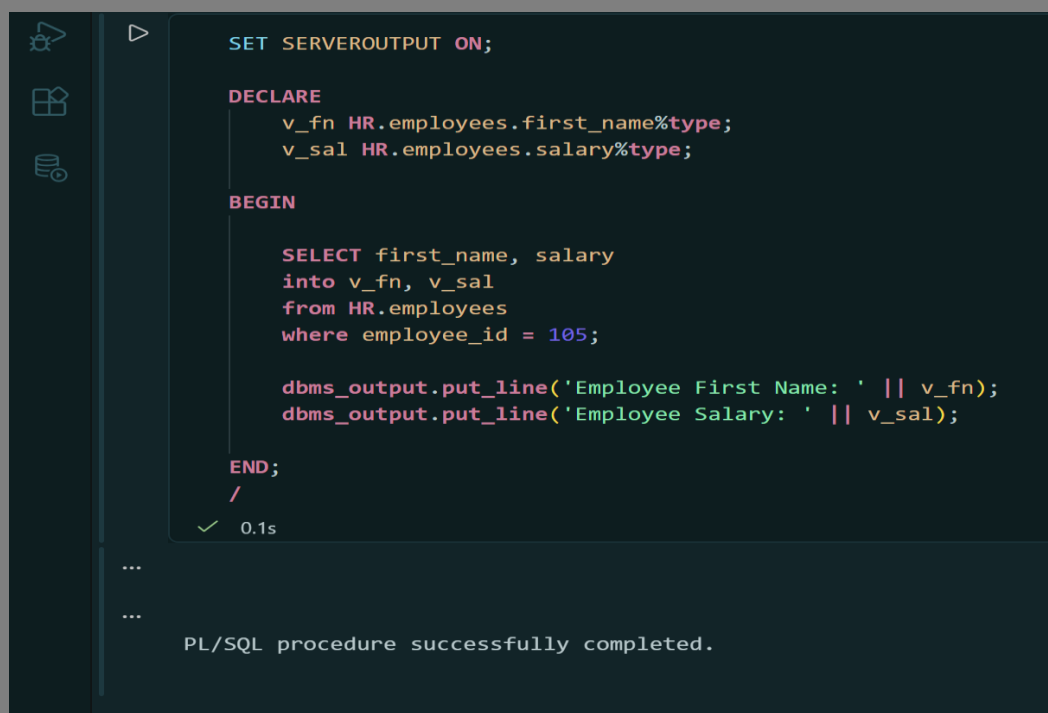
```
dbms_output.put_line('Employee First Name: ' || v_fn);
```

```
dbms_output.put_line('Employee Salary: ' || v_sal);
```

```
END;
```

```
/
```

Screenshot:



```
SET SERVEROUTPUT ON;

DECLARE
    v_fn HR.employees.first_name%type;
    v_sal HR.employees.salary%type;

BEGIN

    SELECT first_name, salary
    into v_fn, v_sal
    from HR.employees
    where employee_id = 105;

    dbms_output.put_line('Employee First Name: ' || v_fn);
    dbms_output.put_line('Employee Salary: ' || v_sal);

END;
/

✓ 0.1s

...

...

PL/SQL procedure successfully completed.
```

Q3)

SQL Query Text:

SET SERVEROUTPUT ON;

DECLARE

eid Number := 110;

v_sal **HR**.employees.salary%**type**;

BEGIN

select salary

into v_sal

from hr.employees

where employee_id = eid;

if (v_sal < 8000) **then**

v_sal := v_sal + 1200;

dbms_output.put_line('Salary was <\$8000, Increased by \$1200.');

else

v_sal := v_sal + 400;

dbms_output.put_line('Standardized raise applied. Increased by \$400.');

end if;

update hr.employees

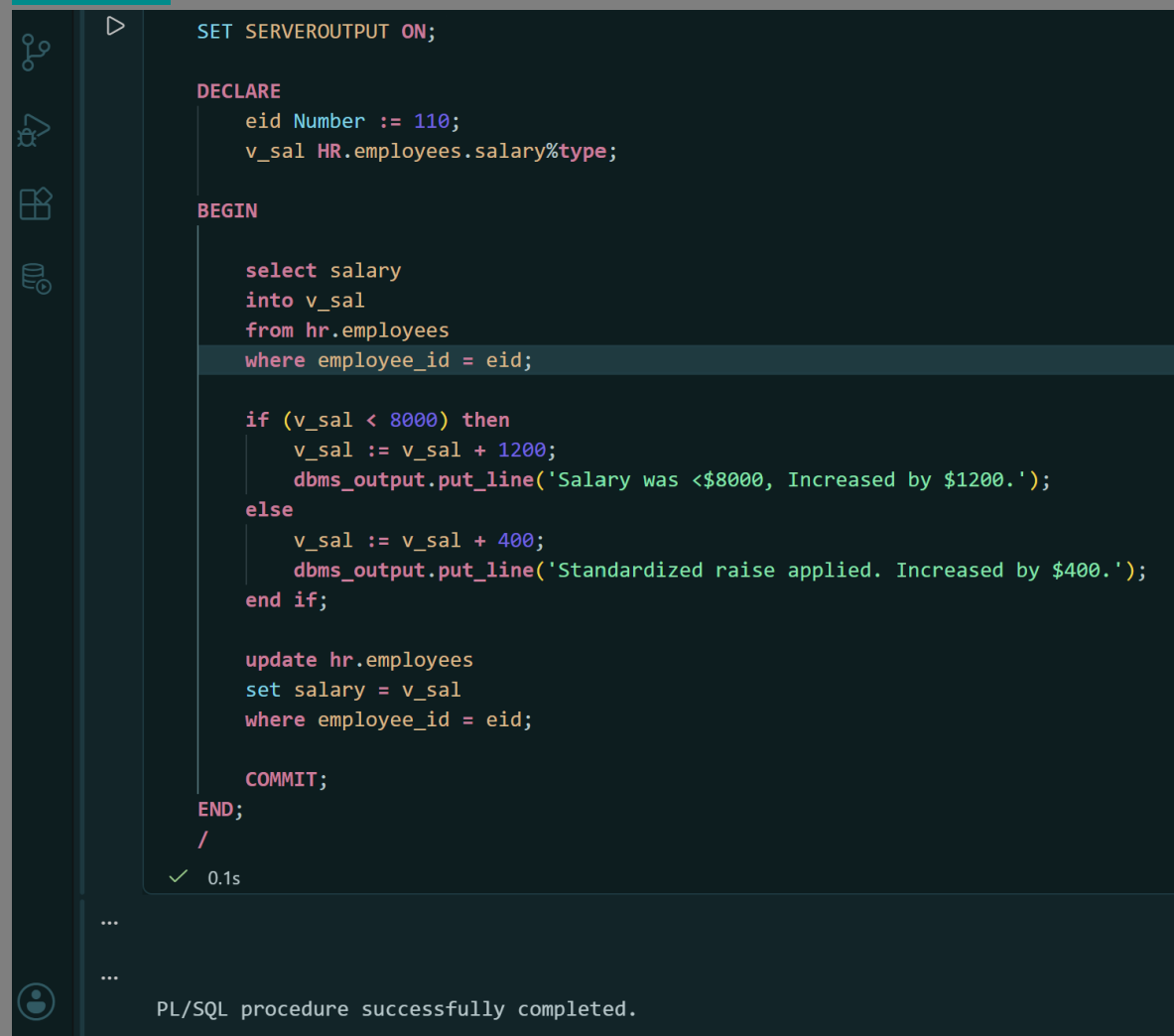
set salary = v_sal

where employee_id = eid;

COMMIT;

END;

/

Screenshot:

The screenshot shows a SQL IDE interface with a dark theme. On the left is a sidebar with icons for a database, a query, a table, and a user. The main area displays a PL/SQL procedure. The code is as follows:

```
SET SERVEROUTPUT ON;

DECLARE
    eid Number := 110;
    v_sal HR.employees.salary%type;

BEGIN

    select salary
    into v_sal
    from hr.employees
    where employee_id = eid;

    if (v_sal < 8000) then
        v_sal := v_sal + 1200;
        dbms_output.put_line('Salary was <$8000, Increased by $1200.');
```

The line `dbms_output.put_line('Salary was <$8000, Increased by $1200.');` is highlighted. Below it, the code continues:

```
    else
        v_sal := v_sal + 400;
        dbms_output.put_line('Standardized raise applied. Increased by $400.');
```

The line `dbms_output.put_line('Standardized raise applied. Increased by $400.');` is highlighted. The code then continues:

```
    end if;

    update hr.employees
    set salary = v_sal
    where employee_id = eid;

    COMMIT;

END;
/
```

Below the code, there is a green checkmark icon and the text "0.1s". At the bottom of the IDE, there is a status bar with a user icon and the text "PL/SQL procedure successfully completed."

Q4)

SQL Query Text:

SET SERVEROUTPUT ON;

DECLARE

eid NUmber := 115;

did HR.EMPLOYEES.DEPARTMENT_ID%TYPE;

v_label VARCHAR2(50);

BEGIN

select department_id

into did

from hr.employees

where employee_id = eid;

v_label := CASE did

WHEN 90 then 'Administration'

WHEN 60 then 'Technical Services'

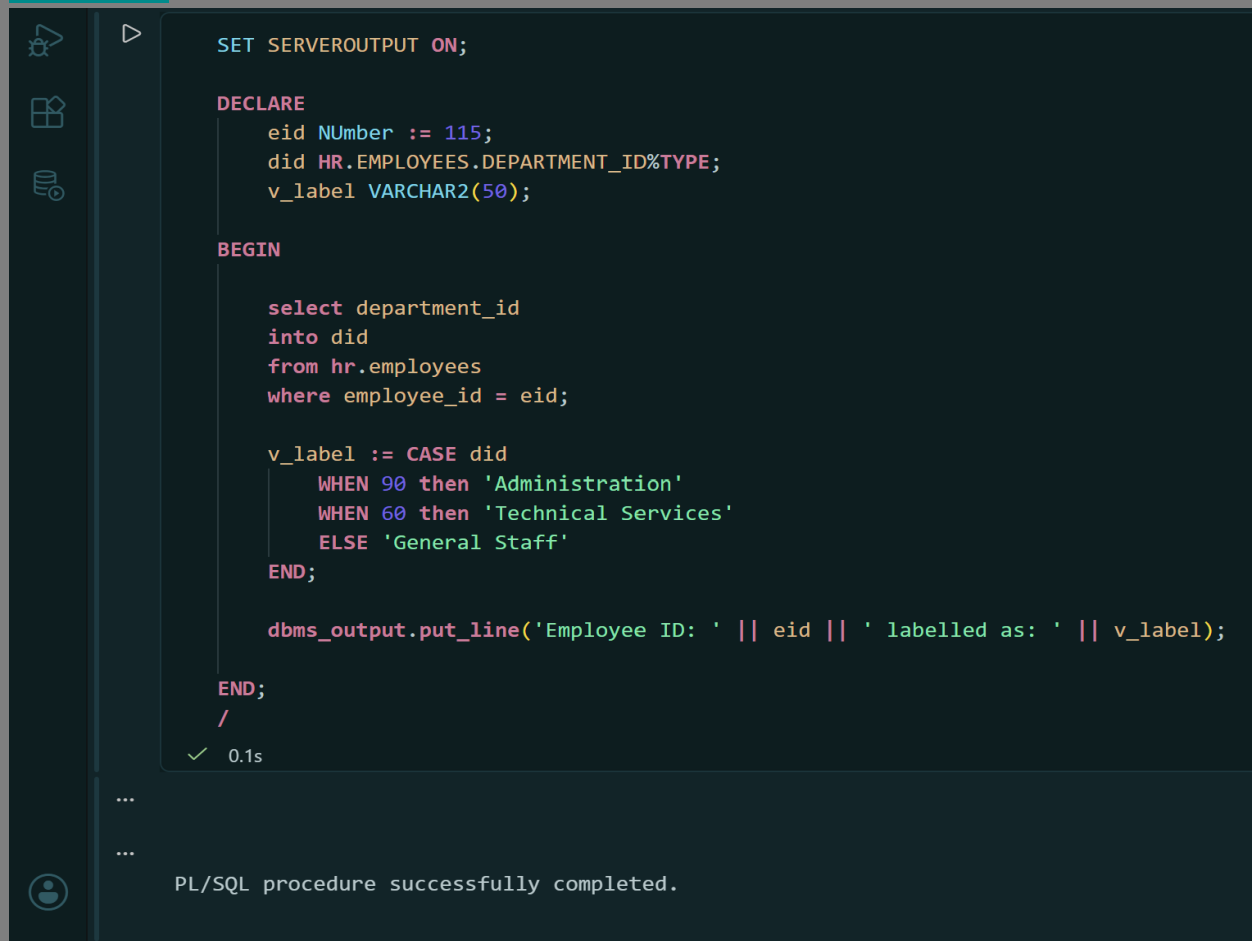
ELSE 'General Staff'

END;

dbms_output.put_line('Employee ID: ' || eid || ' labelled as: ' || v_label);

END;

/

Screenshot:

The screenshot displays the Oracle SQL Developer environment. On the left, a vertical toolbar contains icons for a worksheet, a schema browser, and a database browser. The main workspace is a dark-themed editor showing a PL/SQL procedure. The code is as follows:

```
SET SERVEROUTPUT ON;

DECLARE
    eid NUMBER := 115;
    did HR.EMPLOYEES.DEPARTMENT_ID%TYPE;
    v_label VARCHAR2(50);

BEGIN

    select department_id
    into did
    from hr.employees
    where employee_id = eid;

    v_label := CASE did
        WHEN 90 then 'Administration'
        WHEN 60 then 'Technical Services'
        ELSE 'General Staff'
    END;

    dbms_output.put_line('Employee ID: ' || eid || ' labelled as: ' || v_label);

END;
/
```

Below the code editor, a status bar shows a green checkmark and the text "0.1s". At the bottom of the window, a message pane displays the text "PL/SQL procedure successfully completed." with a user icon on the left.

Q5)

SQL Query Text:

SET SERVEROUTPUT ON;

DECLARE

v Did NUMBER := 90;

v_bonus NUMBER(10, 2);

BEGIN

dbms_output.put_line('Bonus Report for Department ID: 90.');

FOR emp IN (

SELECT first_name, last_name, salary, NVL(commission_pct, 0) AS c_pct

from hr.EMPLOYEES

where department_id = v_Did

) LOOP

v_bonus := 0.00;

if (emp.salary BETWEEN 15000 AND 20000) THEN

v_bonus := (emp.salary * emp.c_pct);

dbms_output.put_line(emp.first_name || ' ' || emp.last_name || ' | Base: \$' || emp.salary || ' | Standard: \$' || v_bonus);

elsif (emp.salary > 20000) THEN

v_bonus := (emp.salary * (emp.c_pct + 0.25));

dbms_output.put_line(emp.first_name || ' ' || emp.last_name || ' | Base: \$' || emp.salary || ' | Standard: \$' || v_bonus);

ELSE

dbms_output.put_line(emp.first_name || ' ' || emp.last_name || ' | Base: \$' || emp.salary || 'No Bonus Applicable.');

end if;

end loop;

EXCEPTION

WHEN NO_DATA_FOUND THEN

DBMS_OUTPUT.PUT_LINE('No employee is in the department ID: 90.');

END;

/

Screenshot:

```
DECLARE
    v_did NUMBER := 90;
    v_bonus NUMBER(10, 2);

BEGIN

    dbms_output.put_line('Bonus Report for Department ID: 90.');
```

```
    FOR emp IN (
        SELECT first_name, last_name, salary, NVL(commission_pct, 0) AS c_pct
        FROM hr.EMPLOYEES
        WHERE department_id = v_did
    ) LOOP

        v_bonus := 0.00;

        IF (emp.salary BETWEEN 15000 AND 20000) THEN
            v_bonus := (emp.salary * emp.c_pct);
            dbms_output.put_line(emp.first_name || ' ' || emp.last_name || ' | Base: $' || emp.salary || ' | Standard: $' || v_bonus);
        ELSEIF (emp.salary > 20000) THEN
            v_bonus := (emp.salary * (emp.c_pct + 0.25));
            dbms_output.put_line(emp.first_name || ' ' || emp.last_name || ' | Base: $' || emp.salary || ' | Standard: $' || v_bonus);
        ELSE
            dbms_output.put_line(emp.first_name || ' ' || emp.last_name || ' | Base: $' || emp.salary || ' | No Bonus Applicable.');
```

```
        END IF;

    END LOOP;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No employee is in the department ID: 90.');
```

```
END;
/
✓ 0.0s
```

...

...

PL/SQL procedure successfully completed.

Q6)

SQL Query Text:

BEGIN

DBMS_OUTPUT.PUT_LINE('--- Shipping Department Employees ---');

FOR emp_rec **IN** (

SELECT first_name, last_name, salary

FROM hr.employees

WHERE department_id = 50

)

LOOP

DBMS_OUTPUT.PUT_LINE('Name: ' || emp_rec.first_name || ' ' || emp_rec.last_name || ' | Salary: \$' || emp_rec.salary);

END LOOP;

END;

/

Screenshot:



```
BEGIN
  DBMS_OUTPUT.PUT_LINE('--- Shipping Department Employees ---');
  FOR emp_rec IN (
    SELECT first_name, last_name, salary
    FROM hr.employees
    WHERE department_id = 50
  )
  LOOP
    DBMS_OUTPUT.PUT_LINE('Name: ' || emp_rec.first_name || ' ' || emp_rec.last_name || ' | Salary: $' || emp_rec.salary);
  END LOOP;
END;
/
✓ 0.0s

...
PL/SQL procedure successfully completed.
```

Q7)

SQL Query Text:

```
SET SERVEROUTPUT ON;
```

```
CREATE OR REPLACE PROCEDURE display_emp_salary (
```

```
  p_emp_id IN NUMBER
```

```
) IS
```

```
  v_salary hr.employees.salary%TYPE;
```

```
BEGIN
```

```
  SELECT salary INTO v_salary
```

```
  FROM hr.employees
```

```
  WHERE employee_id = p_emp_id;
```

```
  DBMS_OUTPUT.PUT_LINE('The current salary for employee ID ' || p_emp_id || ' is $' || v_salary);
```

```
END;
```

```
/
```

```
EXEC display_emp_salary(105);;
```

Screenshot:

```
SET SERVEROUTPUT ON;

CREATE OR REPLACE PROCEDURE display_emp_salary (
    p_emp_id IN NUMBER
) IS
    v_salary hr.employees.salary%TYPE;

BEGIN
    SELECT salary INTO v_salary
    FROM hr.employees
    WHERE employee_id = p_emp_id;

    DBMS_OUTPUT.PUT_LINE('The current salary for employee ID ' || p_emp_id || ' is $' || v_salary);
END;
/

EXEC display_emp_salary(105);
```

Script Output x

Task completed in 0.05 seconds

The current salary for employee ID 105 is \$5280

PL/SQL procedure successfully completed.

Procedure DISPLAY_EMP_SALARY compiled

The current salary for employee ID 105 is \$5280

Dbms Output

Buffer Size: 20000

homeuser x

The current salary for employee ID 105 is \$5280

Q8)

SQL Query Text:

```
SET SERVEROUTPUT ON;
CREATE OR REPLACE PROCEDURE get_emp_dept (
    p_emp_id IN NUMBER,
    p_dept_id OUT NUMBER
) IS
BEGIN
    SELECT department_id INTO p_dept_id
    FROM hr.employees
    WHERE employee_id = p_emp_id;
END;
/

DECLARE
    v_dept_id NUMBER;
BEGIN
    get_emp_dept(105, v_dept_id);
    DBMS_OUTPUT.PUT_LINE('The retrieved Department ID is: ' || v_dept_id);
END;
/
```

Screenshot:

The screenshot displays the Oracle SQL Developer environment. The main editor window contains a PL/SQL script with the following code:

```
SET SERVEROUTPUT ON;
CREATE OR REPLACE PROCEDURE get_emp_dept (
    p_emp_id IN NUMBER,
    p_dept_id OUT NUMBER
) IS
BEGIN
    SELECT department_id INTO p_dept_id
    FROM hr.employees
    WHERE employee_id = p_emp_id;
END;
/
DECLARE
    v_dept_id NUMBER;
BEGIN
    get_emp_dept(105, v_dept_id);
    DBMS_OUTPUT.PUT_LINE('The retrieved Department ID is: ' || v_dept_id);
END;
/
```

Below the script editor, the 'Script Output' window is visible, showing the execution results:

Task completed in 0.103 seconds

Procedure GET_EMP_DEPT compiled

The retrieved Department ID is: 60

PL/SQL procedure successfully completed.

At the bottom, the 'Dbms Output' window is also shown, displaying the same output message:

The retrieved Department ID is: 60

Q9)

SQL Query Text:

```
SET SERVEROUTPUT ON;
CREATE OR REPLACE PROCEDURE increase_by_25_percent (
    p_value IN OUT NUMBER
) IS
BEGIN
    p_value := p_value * 1.25;
END;
/

DECLARE
    v_my_number NUMBER := 200;
BEGIN
    DBMS_OUTPUT.PUT_LINE('Original Value: ' || v_my_number);
    increase_by_25_percent(v_my_number);
    DBMS_OUTPUT.PUT_LINE('Updated Value (After Procedure): ' || v_my_number);
END;
/
```

Screenshot:

The screenshot displays the Oracle SQL Developer environment. The main window shows a PL/SQL script with the following code:

```
SET SERVEROUTPUT ON;  
CREATE OR REPLACE PROCEDURE increase_by_25_percent (  
    p_value IN OUT NUMBER  
) IS  
BEGIN  
    p_value := p_value * 1.25;  
END;  
/  
  
DECLARE  
    v_my_number NUMBER := 200;  
BEGIN  
    DBMS_OUTPUT.PUT_LINE('Original Value: ' || v_my_number);  
    increase_by_25_percent(v_my_number);  
    DBMS_OUTPUT.PUT_LINE('Updated Value (After Procedure): ' || v_my_number);  
END;  
/
```

Below the script editor, the 'Script Output' window is visible, showing the execution results:

Task completed in 0.054 seconds

Procedure INCREASE_BY_25_PERCENT compiled

Original Value: 200
Updated Value (After Procedure): 250

PL/SQL procedure successfully completed.

At the bottom, the 'Dbms Output' window is also visible, showing the same output:

Original Value: 200
Updated Value (After Procedure): 250

Q10)

SQL Query Text:

```
SET SERVEROUTPUT ON;
CREATE OR REPLACE PROCEDURE add_new_location (
    p_street_address IN VARCHAR2,
    p_city IN VARCHAR2
) IS
    v_next_loc_id hr.locations.location_id%TYPE;
BEGIN
    SELECT NVL(MAX(location_id), 0) + 100
    INTO v_next_loc_id
    FROM hr.locations;

    INSERT INTO hr.locations (location_id, street_address, city)
    VALUES (v_next_loc_id, p_street_address, p_city);

    DBMS_OUTPUT.PUT_LINE('Success: New location added in ' || p_city || ' with Location ID: ' ||
v_next_loc_id);
    COMMIT;
END;
/
EXEC add_new_location('123 Sunset Boulevard', 'Karachi');
```

Screenshot:

The screenshot displays the Oracle SQL Developer interface. At the top, the 'Worksheet' tab is active, showing a PL/SQL procedure named 'add_new_location'. The procedure takes two parameters: 'p_street_address' and 'p_city', both of type 'VARCHAR2'. It declares a local variable 'v_next_loc_id' of type 'hr.locations.location_id%TYPE'. The procedure logic includes a 'BEGIN' block with a 'SELECT' statement to find the next available location ID (MAX(location_id) + 100), an 'INSERT' statement to add the new location, and a 'DBMS_OUTPUT.PUT_LINE' statement to display the success message. The procedure ends with 'END;' and is executed using 'EXEC add_new_location('123 Sunset Boulevard', 'Karachi');'.

Below the worksheet, the 'Script Output' window shows the execution results. It indicates that the procedure 'ADD_NEW_LOCATION' was compiled successfully. The output message is 'Success: New location added in Karachi with Location ID: 3301'. The task completed in 0.047 seconds.

At the bottom, the 'Dbms Output' window shows the same success message: 'Success: New location added in Karachi with Location ID: 3301'. The user 'homeuser' is logged in.

```
SET SERVEROUTPUT ON;
CREATE OR REPLACE PROCEDURE add_new_location (
  p_street_address IN VARCHAR2,
  p_city IN VARCHAR2
) IS
  v_next_loc_id hr.locations.location_id%TYPE;
BEGIN
  SELECT NVL(MAX(location_id), 0) + 100
  INTO v_next_loc_id
  FROM hr.locations;

  INSERT INTO hr.locations (location_id, street_address, city)
  VALUES (v_next_loc_id, p_street_address, p_city);

  DBMS_OUTPUT.PUT_LINE('Success: New location added in ' || p_city || ' with Location ID: ' || v_next_loc_id);
  COMMIT;
END;
/
EXEC add_new_location('123 Sunset Boulevard', 'Karachi');
```

Script Output x

Task completed in 0.047 seconds

Procedure ADD_NEW_LOCATION compiled

Success: New location added in Karachi with Location ID: 3301

PL/SQL procedure successfully completed.

Dbms Output

Buffer Size: 20000

homeuser x

Success: New location added in Karachi with Location ID: 3301